

19 Efficient calculation of the square root and inverse square root of a matrix

Jul By Su Jianlin | July 19, 2025 | 25,316 readers | [Kimi](#) (Citation,

set up $\mathbf{P} \in \mathbb{R}^{n \times n}$ It is a function whose eigenvalues are all non-negative real numbers. n This article will discuss the square root of a square matrix of order 1. $\mathbf{P}^{1/2}$ and inverse square root $\mathbf{P}^{-1/2}$ The calculation.

Basic Concepts

matrix $\mathbf{P}$ The square root refers to the property that satisfies $\mathbf{X}^2 = \mathbf{P}$ matrix $\mathbf{X}$ We know that every positive number has two square roots, so it's easy to imagine that the square roots of a matrix are generally not unique either. However, the "arithmetic square root" is unique; the arithmetic square root of a positive number is the positive square root. Similarly, we will... $\mathbf{P}$ The square root of a matrix whose eigenvalues are all non-negative is called the arithmetic square root. This article refers to the arithmetic square root as the matrix square root.

The calculations in this article rely on the matrix symbolic functions we discussed in "What can the matrix symbolic function `mcsign` compute?" :

$$\text{mcsign}(\mathbf{M}) = (\mathbf{M}^2)^{-1/2} \mathbf{M} = \mathbf{M}(\mathbf{M}^2)^{-1/2} \quad (1)$$

Simply put, it is to take any matrix $\mathbf{M} \in \mathbb{R}^{n \times n}$ The eigenvalues are transformed into a new matrix corresponding to the sign function. Assume... $\mathbf{M}$ The eigenvalues of are all real numbers, then `mcsign` Efficient computation can be achieved through the Newton-Schulz iteration:

$$\mathbf{X}_0 = \frac{\mathbf{M}}{\sqrt{\text{tr}(\mathbf{M}^2)}}, \quad \mathbf{X}_{t+1} = a_t + \mathbf{X}_t + b_t + \mathbf{X}_t^3 + c_t + \mathbf{X}_t^5 \quad (2)$$

in $\frac{\mathbf{M}}{\sqrt{\text{tr}(\mathbf{M}^2)}}$ In order to $\mathbf{X}_0$ The eigenvalues are all scaled to $[-1, 1]$ inside, and a_t, b_t, c_t These are the coefficients derived from "Newton-Schulz Iteration of the `msign` Operator (Part 2)" :

t	$a \times 1.01$	$b \times 1.01^3$	$c \times 1.01^5$
1	8.28721	-23.5959	17.3004
2	4.10706	-2.94785	0.544843
3	3.94869	-2.9089	0.551819
4	3.31842	-2.48849	0.510049
5	2.30065	-1.6689	0.418807
6	1.8913	1.268	0.376804
7	1.875	-1.25	0.375
8	1.875	-1.25	0.375

In fact, when $\mathbf{M}$ When all the eigenvalues are real numbers, `mcsign` The calculation principle and another matrix symbolic function `msign` They are interconnected.

Calculation Principles

The next calculation starts with the identity.

$$\text{mcsgn}\left(\begin{bmatrix} \mathbf{0} & \mathbf{A} \\ \mathbf{B} & \mathbf{0} \end{bmatrix}\right) = \begin{bmatrix} \mathbf{0} & \mathbf{A}(\mathbf{BA})^{-1/2} \\ \mathbf{B}(\mathbf{AB})^{-1/2} & \mathbf{0} \end{bmatrix} \quad (3)$$

Direct substitution [mcsgn](#) The definition can verify that the above formula holds true (Note: $\mathbf{A}$, $\mathbf{B}$ (It may not be a square matrix). Next, we need to determine under what circumstances the left side is calculated. [mcsgn](#) The eigenvalues of the matrix are all real numbers. Let... λ It is one of its non-zero eigenvalues, then

$$0 = \det\left(\lambda \mathbf{I} - \begin{bmatrix} \mathbf{0} & \mathbf{A} \\ \mathbf{B} & \mathbf{0} \end{bmatrix}\right) = \det\left(\begin{bmatrix} \lambda \mathbf{I} & -\mathbf{A} \\ -\mathbf{B} & \lambda \mathbf{I} \end{bmatrix}\right) = \det(\lambda^2 \mathbf{I} - \mathbf{AB}) \quad (4)$$

Right now λ^2 It is a matrix $\mathbf{AB}$ The eigenvalues of the above block matrix. This means that all eigenvalues of the above block matrix are real numbers if and only if $\mathbf{AB}$ All eigenvalues are non-negative.

Iterating directly over the original matrix is possible, but computationally expensive. We can utilize its anti-diagonal structure to reduce computation. Because...

$$\begin{bmatrix} \mathbf{0} & \mathbf{Y} \\ \mathbf{Z} & \mathbf{0} \end{bmatrix}^3 = \begin{bmatrix} \mathbf{0} & (\mathbf{YZ})\mathbf{Y} \\ \mathbf{Z}(\mathbf{YZ}) & \mathbf{0} \end{bmatrix}, \quad \begin{bmatrix} \mathbf{0} & \mathbf{Y} \\ \mathbf{Z} & \mathbf{0} \end{bmatrix}^5 = \begin{bmatrix} \mathbf{0} & (\mathbf{YZ})^2\mathbf{Y} \\ \mathbf{Z}(\mathbf{YZ})^2 & \mathbf{0} \end{bmatrix} \quad (5)$$

We can obtain iterations

$$\mathbf{Y}_{t+1} = (a_{t+1}\mathbf{I} + b_{t+1}\mathbf{Y}_t\mathbf{Z}_t + c_{t+1}(\mathbf{Y}_t\mathbf{Z}_t)^2)\mathbf{Y}_t \quad (6)$$

$$\mathbf{Z}_{t+1} = \mathbf{Z}_t(a_{t+1}\mathbf{I} + b_{t+1}\mathbf{Y}_t\mathbf{Z}_t + c_{t+1}(\mathbf{Y}_t\mathbf{Z}_t)^2) \quad (7)$$

So $\mathbf{Y}_t \rightarrow \mathbf{A}(\mathbf{BA})^{-1/2}$, $\mathbf{Z}_t \rightarrow \mathbf{B}(\mathbf{AB})^{-1/2}$ Specifically, multiplying the two equations above yields... $\mathbf{Y}_t\mathbf{Z}_t$ recursion

$$\mathbf{Y}_{t+1}\mathbf{Z}_{t+1} = (a_{t+1}\mathbf{I} + b_{t+1}\mathbf{Y}_t\mathbf{Z}_t + c_{t+1}(\mathbf{Y}_t\mathbf{Z}_t)^2)^2\mathbf{Y}_t\mathbf{Z}_t \quad (8)$$

Find the square root .

Now we will formally proceed with the calculation of the square root. Because of the assumptions made... $\mathbf{P}$ The eigenvalues are non-negative, so we can always solve them by dividing by... $\text{tr}(\mathbf{P})$ Further compress its eigenvalues to $0 \sim 1$ Between these, and without loss of generality, we assume $\mathbf{P}$ The eigenvalues are all $[0, 1]$ This allows for direct calculation using the Newton-Schulz iterative method. [mcsgn](#) Alright.

Will $\mathbf{A} = \mathbf{P}$, $\mathbf{B} = \mathbf{I}$ Substitute into formula (3) You can get

$$\text{mcsgn}\left(\begin{bmatrix} \mathbf{0} & \mathbf{P} \\ \mathbf{I} & \mathbf{0} \end{bmatrix}\right) = \begin{bmatrix} \mathbf{0} & \mathbf{P}^{1/2} \\ \mathbf{P}^{-1/2} & \mathbf{0} \end{bmatrix} \quad (9)$$

It's amazing, theoretically it only takes...[mcsn](#)In one go, both the square root and the inverse square root can be found, that is, according to the formula...[\(6\)](#)and[\(7\)](#)Through iteration, we can accomplish two tasks simultaneously!

However, the reality is not so ideal. If P If there are singular values very close to 0, then... $P^{-1/2}$ It can cause a numerical explosion (equivalent to the occurrence of...) $1/\sqrt{0}$),but $P^{1/2}$ No, so let's assume we only care about $P^{1/2}$ The value is calculated simultaneously. $P^{1/2}$, $P^{-1/2}$ This would actually increase numerical instability. A better approach in this case is through the formula...[\(6\)](#)and[\(8\)](#)Iterate, only calculate $P^{1/2}$:

$$Y_0 = P, \quad Y_0 Z_0 = P$$

$$Y_{t+1} = (a_{t+1}I + b_{t+1}Y_t Z_t + c_{t+1}(Y_t Z_t)^2)Y_t \quad (10)$$

$$Y_{t+1} Z_{t+1} = (a_{t+1}I + b_{t+1}Y_t Z_t + c_{t+1}(Y_t Z_t)^2)^2 Y_t Z_t \quad (11)$$

$$\lim_{t \rightarrow \infty} Y_t = P^{1/2}$$

because Z_t The limit is $P^{-1/2}$,so $Y_t Z_t$ The limit is I Therefore, iteration $Y_t Z_t$ It is less prone to numerical risks. The sample code is as follows:

```

1  import numpy as np
2
3  def abc(steps):
4      coefs = [
5          (8.287212018145622, -23.59588651909882, 17.300387312530923),
6          (4.107059111542197, -2.9478499167379084, 0.54484310829266),
7          (3.9486908534822938, -2.908902115962947, 0.5518191394370131),
8          (3.3184196573706055, -2.488488024314878, 0.5100489401237208),
9          (2.3006520199548186, -1.6689039845747518, 0.4188073119525678),
10         (1.8913014077874002, -1.2679958271945908, 0.37680408948524996),
11         (1.875, -1.25, 0.375)
12     ]
13     for a, b, c in coefs[:steps] + max(steps - 7, 0) * coefs[-1:]:
14         yield a / 1.01, b / 1.01**3, c / 1.01**5
15
16 def msqrt(P, steps=6):
17     Y = YZ = P / (t := np.trace(P))
18     I = np.eye(P.shape[0])
19     for a, b, c in abc(steps):
20         W = a * I + b * YZ + c * YZ @ YZ
21         Y, YZ = W @ Y, W @ W @ YZ
22     return Y * t**0.5
23
24 d = 100
25 P = (x := np.random.randn(d, d) / d**0.5) @ x.T
26 np.abs(msqrt(P) @ msqrt(P) - P).mean() # ~ 2e-4

```

Inverse square root

If we must explicitly find the inverse square root $P^{-1/2}$ Then there's not much you can do; what's destined to explode will explode eventually, regardless of the method. (7), (6) Combination or formula (7), (8) The effects of combining them should be similar, but the latter should be relatively more stable.

$$Z_0 = I, \quad Y_0 Z_0 = P$$

$$Z_{t+1} = Z_t(a_{t+1}I + b_{t+1}Y_t Z_t + c_{t+1}(Y_t Z_t)^2) \quad (12)$$

$$Y_{t+1} Z_{t+1} = (a_{t+1}I + b_{t+1}Y_t Z_t + c_{t+1}(Y_t Z_t)^2)^2 Y_t Z_t \quad (13)$$

$$\lim_{t \rightarrow \infty} Z_t = P^{-1/2}$$

The following is a sample code:

```
1  def mrsqrt(P, steps=6):
2      YZ = P / (t := np.trace(P))
3      Z = I = np.eye(P.shape[0])
4      for a, b, c in abc(steps):
5          W = a * I + b * YZ + c * YZ @ YZ
6          Z, YZ = Z @ W, W @ W @ YZ
7      return Z / t**0.5
8
9  d = 100
10 P = (x := np.random.randn(d, d) / d**0.5) @ x.T
11 np.abs(mrsqrt(P) @ mrsqrt(P) @ P - np.eye(d)).mean() # ~= 5e-4
```

Matrix multiplication

However, most of the time, seeking $P^{-1/2}$ This is just an intermediate step; after calculating it, it usually needs to be multiplied by another matrix. Let the matrix be... $G \in \mathbb{R}^{m \times n}$ We need to calculate $GP^{-1/2}$ If we can $GP^{-1/2}$ As a whole iterable object, compared to calculating it separately... $P^{-1/2}$ Then performing matrix multiplication often results in better numerical stability.

Let us observe the formula carefully. (12) and (13) It is not difficult to see that when we put $Y_t Z_t$ When considered as a whole, its iterative form (13) Actually independent of Z_t Therefore, Z_t The form (12) Essentially, it's just a linear recursion! Multiplying it by a matrix on the left doesn't change the iteration form; we only need to modify the initi

al value, thus obtaining...

$$\mathbf{Z}_0 = \mathbf{G}, \quad \mathbf{Y}_0 \mathbf{Z}_0 = \mathbf{P}$$

$$\mathbf{Z}_{t+1} = \mathbf{Z}_t(a_{t+1}\mathbf{I} + b_{t+1}\mathbf{Y}_t\mathbf{Z}_t + c_{t+1}(\mathbf{Y}_t\mathbf{Z}_t)^2) \quad (14)$$

$$\mathbf{Y}_{t+1}\mathbf{Z}_{t+1} = (a_{t+1}\mathbf{I} + b_{t+1}\mathbf{Y}_t\mathbf{Z}_t + c_{t+1}(\mathbf{Y}_t\mathbf{Z}_t)^2)^2\mathbf{Y}_t\mathbf{Z}_t \quad (15)$$

$$\lim_{t \rightarrow \infty} \mathbf{Z}_t = \mathbf{G}\mathbf{P}^{-1/2}$$

Reference code:

```

1  import scipy as sp
2
3  def matmul_mrsqrt(G, P, steps=6):
4      YZ = P / (t := np.trace(P))
5      Z, I = G, np.eye(P.shape[0])
6      for a, b, c in abc(steps):
7          W = a * I + b * YZ + c * YZ @ YZ
8          Z, YZ = Z @ W, W @ W @ YZ
9      return Z / t**0.5
10
11  d = 100
12  P = (x := np.random.randn(d, d) / d**0.5) @ x.T
13  G = np.random.randn(2 * d, d) / d**0.5
14  X = matmul_mrsqrt(G, P)
15  np.abs(X @ sp.linalg.sqrtm(P) - G).mean() # ~= 1e-4

```

Now, let's look back at the algorithm for finding the square root. It's not hard to see that it is actually... $\mathbf{G} = \mathbf{P}$ An other equivalent way to write the iteration in this section is, $\mathbf{P}^{1/2} = \mathbf{P}\mathbf{P}^{-1/2}$ Therefore, although we appear to have separated the discussion of the three iterations into three sections, they are essentially special cases of the last iteration!

Ultimate Promotion

Finally, we can also extend it to $\mathbf{Q}^{-1/2}\mathbf{G}\mathbf{P}^{-1/2}$ The calculation, in which $\mathbf{Q} \in \mathbb{R}^{m \times m}$ This is another matrix with non-negative eigenvalues, and the result is as follows:

$$\mathbf{G}_0 = \mathbf{G}, \quad \mathbf{Q}_0 = \mathbf{Q}, \quad \mathbf{P}_0 = \mathbf{P}$$

$$\mathbf{G}_{t+1} = (a_{t+1}\mathbf{I} + b_{t+1}\mathbf{Q}_t + c_{t+1}\mathbf{Q}_t^2)\mathbf{G}_t(a_{t+1}\mathbf{I} + b_{t+1}\mathbf{P}_t + c_{t+1}\mathbf{P}_t^2) \quad (16)$$

$$\mathbf{Q}_{t+1} = (a_{t+1}\mathbf{I} + b_{t+1}\mathbf{Q}_t + c_{t+1}\mathbf{Q}_t^2)^2\mathbf{Q}_t \quad (17)$$

$$\mathbf{P}_{t+1} = (a_{t+1}\mathbf{I} + b_{t+1}\mathbf{P}_t + c_{t+1}\mathbf{P}_t^2)^2\mathbf{P}_t \quad (18)$$

$$\lim_{t \rightarrow \infty} \mathbf{G}_t = \mathbf{Q}^{-1/2}\mathbf{G}\mathbf{P}^{-1/2}$$

Reference code:

```
1 def mrsqrt_matmul_mrsqrt(Q, G, P, steps=6):
2     Q = Q / (t1 := np.trace(Q))
3     P = P / (t2 := np.trace(P))
4     I1, I2 = np.eye(Q.shape[0]), np.eye(P.shape[0])
5     for a, b, c in abc(steps):
6         W1 = a * I1 + b * Q + c * Q @ Q
7         W2 = a * I2 + b * P + c * P @ P
8         G, Q, P = W1 @ G @ W2, W1 @ W1 @ Q, W2 @ W2 @ P
9     return G / (t1 * t2) **0.5
10
11 d = 100
12 Q = (x := np.random.randn(2 * d, 2 * d) / (2 * d)**0.5) @ x.T
13 P = (x := np.random.randn(d, d) / d**0.5) @ x.T
14 G = np.random.randn(2 * d, d) / d**0.5
15 X = mrsqrt_matmul_mrsqrt(Q, G, P)
16 np.abs(sp.linalg.sqrtn(Q) @ X @ sp.linalg.sqrtn(P) - G).mean() # ~ 2e-3
```

Readers are encouraged to complete the proof themselves based on the results from the previous sections.

For the Shampoo optimizer, we need to calculate... $Q^{-1/4}GP^{-1/4}$ The most feasible solution at present is to first calculate... $Q^{1/2}$ and $P^{1/2}$ Then substitute it into the above iteration to find $(Q^{1/2})^{-1/2}G(P^{1/2})^{-1/2}$ While the computational load appears large, in reality, computing power is often not the bottleneck during the Optimizer Update phase. As long as the algorithm can be fully parallelized, the time increase will not be significant. $Q^{1/2}$ and $P^{1/2}$ The computation can be parallelized, and the two W1 and W2 can also be parallelized during the iteration process, so it should still be acceptable.

Of course, it is definitely slower than Muon, after all, Shampoo increases complexity so much, it can't be done without any cost (see "Efficient Calculation of the r-th Root and Inverse r-th Root of a Matrix" for later).

Article Summary

This paper proposes a method to transform the square root and inverse square root of a matrix into...mcsgn The form utilizes its Newton-Schulz iteration to achieve efficient computation.

Please include this article's URL when reprinting: <https://kexue.fm/archives/11158>

For more detailed information on reprinting, please refer to: "Science Space FAQ"

If you need to cite this article, please refer to:

Su Jianlin. (Jul. 19, 2025). "Efficient Calculation of Matrix Square Root and Inverse Square Root" [Blog post]. Retrieved from <https://kexue.fm/archives/11158>

```
@online{kexuefm-11158,  
  title={Efficient Calculation of Matrix Square Root and Inverse Square Root},  
  author={苏剑林},  
  year={2025},  
  month={Jul},  
  url={\url{https://kexue.fm/archives/11158}},  
}
```